

NutNet multidimensional stability analyses

05: Reproduce remaining supplementary figures and sensitivity analyses

Qiang Yang

2026-06-22

Overview

This document reproduces the remaining supplementary figures and sensitivity analyses associated with the NutNet multidimensional stability manuscript. These analyses complement the main workflows for data preparation, stability calculation, single-stability modelling, and stability dimensionality. They focus on supporting figures related to legume relative abundance, species richness, dominance, phosphorus responses along baseline soil N:P gradients, directional biomass responses, site-level enrichment-intensity maps, and a sensitivity analysis using only the subset of plots included in the dimensionality analysis.

Each major figure is divided into smaller subsections that separate data preparation, model fitting, coefficient extraction, prediction, and plotting. This structure is intended to make the workflow easier to inspect and follow. Some data-preparation steps are repeated intentionally so that each figure section is relatively self-contained and does not depend on hidden objects created many pages earlier.

Output

Running this workflow displays the following outputs in the knitted document:

1. A figure showing treatment effects on the relative abundance of legumes through time (Fig. S6).
2. A figure showing treatment effects on plant species richness through time (Fig. s4).
3. A figure showing treatment effects on the proportional dominance of dominant species through time (Fig. s5).
4. A figure showing whether stability responses to phosphorus enrichment are related to baseline soil N:P ratios (Fig. s7).

5. A model-output table and a figure showing directional biomass responses, measured as log response ratios, to nitrogen and phosphorus enrichment intensity through time (Fig. s3).
6. Maps showing site-level baseline nutrient concentrations and relative enrichment intensities for nitrogen, phosphorus, and potassium (Fig. s1).
7. A sensitivity-analysis figure testing whether the main single-stability-response results are qualitatively robust when analyses are restricted to the subset of plots used in the dimensionality analysis (Fig. s10).

```
knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE
)

# Required packages for the remaining figure and sensitivity analyses.
Packages <- c(
  "tidyverse", "magrittr", "grid", "gridExtra", "rnatualearth",
  "rnatualearthdata", "ggcorrplot", "scales", "sp", "sf", "broom",
  "broom.mixed", "nlme", "scico", "vegan", "dlookr", "GGally",
  "PerformanceAnalytics", "glmmTMB", "ggeffects", "patchwork",
  "RColorBrewer", "lme4", "lmerTest", "cowplot", "gtable", "rgbif"
)
pacman::p_load(char = Packages)

# Avoid ambiguity among functions with the same names in different packages.
filter <- dplyr::filter
select <- dplyr::select
summarise <- dplyr::summarise

# Working paths used throughout the project.
main.path <- getwd()
data.path <- paste0(main.path, "/mydata")
figure.path <- paste0(main.path, "/myfigure")

set.seed(1234)
```

1. Community composition and functional-group responses

This section reproduces supplementary analyses of plant community changes that help interpret stability responses. It includes three related analyses: the relative abundance of legumes, species richness, and the proportional share of dominant species. These responses are modelled through time as a function of nutrient-addition treatment while accounting for site-level differences using random intercepts.

1.1 Legume relative abundance

This analysis tests whether the relative abundance of legumes changes through time under nitrogen, phosphorus, and potassium enrichment. Legume cover is calculated from species-level cover data after identifying Fabaceae taxa and manually resolving a small number of unclear taxonomic assignments.

1.1.1 Select plots and extract plant taxa

The analysis first selects treatment plots for which compositional stability could be calculated and then adds the corresponding control plots from the same experimental blocks.

```
# Reload the datasets required for this figure.
load(file = paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(file = paste0(data.path, "/df.tidy.Rdata"))

# Prepare data for the legume relative-abundance analysis.
# This analysis evaluates whether legumes respond differently through time among nutrient treatments.
# Select treatment plots for which compositional-change values were calculated.
df_plot_selected <- df.tidy.stabilities %>%
  filter(!is.na(comp_change)) %>%
  select(site_code, block, trt, plot) %>%
  distinct()

# Add the corresponding control plots from the same blocks.
df_plot_control <- df_plot_selected %>%
  select(site_code, block) %>%
  distinct() %>%
  left_join(
    df.tidy %>%
      filter(trt == "Control") %>%
      select(site_code, block, plot, trt) %>%
      distinct()
```

```

)
df_plot_selected <- df_plot_selected %>%
  bind_rows(df_plot_control)

df_plot_selected <- df_tidy %>%
  select(site_code, block, plot, trt, coverage.data) %>%
  semi_join(df_plot_selected)
# Extract the list of plant taxa represented in the selected plots.
species_list <- df_plot_selected %>%
  select(coverage.data) %>%
  unnest(cols = coverage.data) %>%
  select(Taxon) %>%
  unlist() %>%
  unique()
# Use the GBIF taxonomic backbone to identify taxa belonging to Fabaceae.

family_backbone <- sapply(species_list, function(x){
  out <- name_backbone(name = x)$family
  ifelse(is.null(out), "NA", out)
})
legume_list1 <- species_list[which(family_backbone == "Fabaceae")]

# A small number of taxa could not be assigned to family automatically.

# Use a manually checked file to resolve these remaining taxa.
df_manual_check <- read_csv(paste0(data.path,
                                   "/legumes_status_a_few_unclear_species.csv"))
legume_list2 <- df_manual_check %>%
  filter(legume == "legume") %>%
  pull(name)
legume_list <- union(legume_list1, legume_list2)

```

1.1.2 Calculate legume relative abundance and fit the model

For each plot and year, species cover is classified as legume or non-legume cover. Legume relative abundance is then analysed with a beta generalized linear mixed-effects model with treatment, year, and their interaction as fixed effects and site as a random intercept.

```

cal_legume_rel <- function(dt) {
  dt %>%
    mutate(
      is_legume = if_else(Taxon %in% legume_list, "legume", "non_legume")
    ) %>%
    group_by(year_trt, is_legume) %>%
    summarise(cover = sum(max_cover, na.rm = TRUE), .groups = "drop") %>%

    # Make sure both legume and non_legume exist for every year_trt
    complete(
      year_trt,
      is_legume = c("legume", "non_legume"),
      fill = list(cover = 0)
    ) %>%

    pivot_wider(
      names_from = is_legume,
      values_from = cover,
      values_fill = 0
    ) %>%

    mutate(
      total_cover = legume + non_legume,
      rel_abundance_legume = if_else(
        total_cover > 0,
        legume / total_cover,
        NA_real_
      )
    ) %>%
    select(year_trt, rel_abundance_legume)
}

df_legumes <- df_plot_selected %>%
  select(site_code, block, plot, trt, coverage.data) %>%
  semi_join(df_plot_selected) %>%
  mutate(coverage.data = map(coverage.data, cal_legume_rel))
df_legumes <- df_legumes %>%
  unnest(cols = coverage.data)

```

```
m <- glmmTMB(
  rel_abundance_legume ~ trt * year_trt +
    (1 | site_code),
  data = df_legumes,
  ziformula = ~ 1, # considering zero-inflation
  family = beta_family(link = "logit")
)
```

1.1.3 Build coefficient panel

This panel summarizes the fixed-effect estimates, confidence intervals, and significance direction from the legume model.

```
coefs <- broom.mixed::tidy(m, effects = "fixed", conf.int = TRUE)

# Remove the intercept so that the figure focuses on treatment and temporal effects.
coefs2 <- coefs %>%
  filter(term != "(Intercept)")

# Use readable predictor names in the coefficient panel.
coefs2$term <- recode(coefs2$term,
  "trtK" = "K vs Control",
  "trtN" = "N vs Control",
  "trtP" = "P vs Control",
  "year_trt" = "Year (Control)",
  "trtK:year_trt" = "K x Year",
  "trtN:year_trt" = "N x Year",
  "trtP:year_trt" = "P x Year"
)

# Classify estimates by direction and statistical significance.
coefs2 %<>%
  mutate(sig.sign = map2_chr(estimate, p.value, function(x, p) {
    if (x < 0 & p < 0.05) {
      out <- "negative"
    } else if (x > 0 & p < 0.05) {
      out <- "positive"
    } else {
      out <- "non-significant"
    }
  })
```

```

    }
    return(out)
  })

p_legume_a <- ggplot(coefs2, aes(estimate, term)) +
  geom_point(size = 3, aes(colour = sig.sign)) +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high, colour = sig.sign), height = 0.15) +
  geom_vline(xintercept = 0, linetype = 2, color = "grey40") +
  scale_color_manual(values = c("blue", "grey", "red")) +
  theme_bw(base_size = 14) +
  labs(x = "Coefficient (logit scale)", y = "",
       colour = "Significance: ") +
  theme(axis.text = element_text(size=12, color = "black"),
        legend.position = "bottom")

```

1.1.4 Build prediction panel and display the figure

This panel shows model-predicted legume relative abundance through time for each treatment. The two panels are combined into the final figure.

```

pred <- ggpredict(m, terms = c("year_trt", "trt"))
pred <- as_tibble(pred)

pred <- pred %>%
  mutate(
    trt_lab = factor(
      group,
      levels = c("Control", "N", "P", "K"),      # original names
      labels = c("Control", "+N", "+P", "+K")    # what you want to show
    )
  )

dark2_cols <- brewer.pal(3, "Dark2")
col_vec <- c(
  "Control" = "grey40",
  "+N"      = dark2_cols[1],
  "+P"      = dark2_cols[2],
  "+K"      = dark2_cols[3]
)

```

```

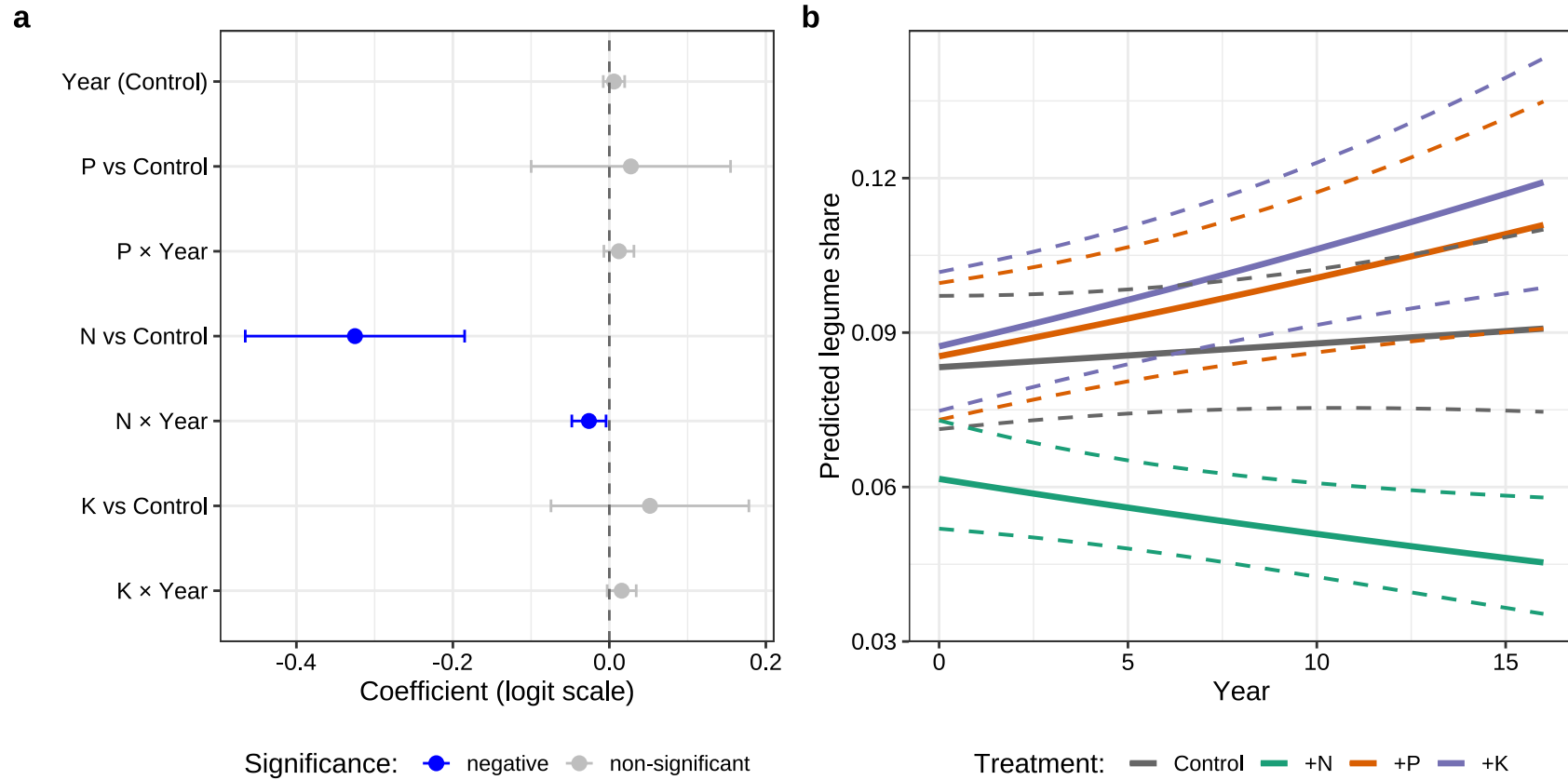
)

p_legume_b <- ggplot(pred) +
  # Predicted mean responses.
  geom_line(aes(x = x, y = predicted,
                colour = trt_lab, group = trt_lab),
            size = 1.5) +
  # Lower confidence limits.
  geom_line(aes(x = x, y = conf.low,
                colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  # Upper confidence limits.
  geom_line(aes(x = x, y = conf.high,
                colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  theme_bw(base_size = 14) +
  labs(
    x = "Year",
    y = "Predicted legume share",
    colour = "Treatment: "
  ) +
  scale_colour_manual(values = col_vec) +
  theme(
    plot.title = element_blank(),
    legend.position = "bottom",
    axis.text = element_text(size=12, color = "black")
  )

p_legume <- p_legume_a + p_legume_b +
  plot_layout(ncol = 2, widths = c(1, 1.2)) +
  plot_annotation(tag_levels = "a") &
  theme(plot.tag = element_text(face = "bold", size = 16))

p_legume

```

1.2 Species richness and dominance

This section quantifies annual species richness and the proportional share of dominant species in each plot. Dominant species are defined at the plot level based on long-term cover rankings, with the number of dominant species adjusted according to the richness of each plot.

1.2.1 Prepare richness and dominance datasets

The code reconstructs the selected plot set and unnests species-cover data. Dominant species are identified using long-term mean cover within each plot, and annual richness and dominance metrics are then calculated for each plot-year.

```
# Select the same treatment plots used for compositional stability calculations,
# and add the matching control plots from the same blocks.
df_plot_selected <- df.tidy.stabilities %>%
  filter(!is.na(comp_change)) %>%
  select(site_code, block, trt, plot) %>%
  distinct()

df_plot_control <- df_plot_selected %>%
  select(site_code, block) %>%
  distinct() %>%
  left_join(
    df.tidy %>%
      filter(trt == "Control") %>%
      select(site_code, block, plot, trt) %>%
      distinct()
  )

df_plot_selected <- df_plot_selected %>%
  bind_rows(df_plot_control)

# Reload the datasets required for this figure.
load(file = paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(file = paste0(data.path, "/df.tidy.Rdata"))

# Prepare species richness and dominance metrics.

# Identify dominant species within each plot based on long-term cover rankings.
df <- df.tidy %>%
  semi_join(df_plot_selected)
df <- df %>%
  select(site_code, block, plot, trt, coverage.data) %>%
  unnest(cols = coverage.data) %>%
  select(site_code, block, plot, trt, year_trt, species = Taxon, cover = max_cover)
dom_ids <- df %>%
```

```

group_by(site_code, block, plot, trt, species) %>%
summarise(mean_cover = mean(cover, na.rm = TRUE), .groups = "drop") %>%
group_by(site_code, block, plot, trt) %>%
arrange(desc(mean_cover), .by_group = TRUE) %>%
mutate(
  richness_plot = n(), # number of species in this plot × trt

  # Rule used to define the number of dominant species per plot:
  # if richness ≤ 2 → 1 dominant
  # if 3-4 → 2 dominants
  # if ≥ 5 → 3 dominants
  n_dom = case_when(
    richness_plot ≤ 2 ~ 1L,
    richness_plot ≤ 4 ~ 2L,
    TRUE ~ 3L
  ),

  is_dominant = row_number() ≤ n_dom
)
# Calculate the relative share of dominant species for each plot-year.
# Species richness is calculated for the same plot-year observations.
df_dom <- df %>%
left_join(dom_ids %>% select(site_code, block, plot, trt, species, is_dominant),
  by = c("site_code", "block", "plot", "trt", "species")) %>%
mutate(is_dominant = ifelse(is.na(is_dominant), FALSE, is_dominant)) %>%
group_by(site_code, block, plot, trt, year_trt) %>%
mutate(
  total = sum(cover, na.rm = TRUE),
  rel = cover / total
) %>%
summarise(
  dom_share = sum(rel[is_dominant], na.rm = TRUE),
  richness = n_distinct(species[cover > 0]),
  .groups = "drop"
)

```

1.2.2 Model and coefficient panel for species richness

Species richness is modelled using a negative-binomial generalized linear mixed-effects model. The coefficient panel summarizes treatment, year, and treatment-by-year effects.

```
## ---- RICHNESS MODEL ----

m_rich <- glmmTMB(
  richness ~ trt * year_trt +
    (1 | site_code),
  data = df_dom,
  family = nbinom2(link = "log")
)

## ---- COEFFICIENT FIGURE (RICHNESS) ----

coefs_rich <- broom.mixed::tidy(m_rich, effects = "fixed", conf.int = TRUE)

coefs_rich2 <- coefs_rich %>%
  filter(term != "(Intercept)")

coefs_rich2$term <- dplyr::recode(
  coefs_rich2$term,
  "trtK" = "K vs Control",
  "trtN" = "N vs Control",
  "trtP" = "P vs Control",
  "year_trt" = "Year (Control)",
  "trtK:year_trt" = "K × Year",
  "trtN:year_trt" = "N × Year",
  "trtP:year_trt" = "P × Year"
)

coefs_rich2 %<>%
  mutate(sig.sign = purrr::map2_chr(estimate, p.value, function(x, p) {
    if (x < 0 & p < 0.05) {
      "negative"
    } else if (x > 0 & p < 0.05) {
      "positive"
    } else {
```

```

    "non-significant"
  }
}))

p_rich_a <- ggplot(coefs_rich2, aes(estimate, term)) +
  geom_point(size = 3, aes(colour = sig.sign)) +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high, colour = sig.sign),
    height = 0.15) +
  geom_vline(xintercept = 0, linetype = 2, color = "grey40") +
  scale_color_manual(values = c("blue", "grey", "red")) +
  theme_bw(base_size = 14) +
  labs(
    x = "Coefficient (log scale)",
    y = "",
    colour = "Significance:"
  ) +
  theme(
    axis.text = element_text(size = 12, color = "black"),
    legend.position = "bottom"
  )

```

1.2.3 Prediction panel for species richness

Model predictions are generated to show how species richness changes through time across treatments.

```

## ---- PREDICTION FIGURE (RICHNESS) ----

pred_rich <- ggpredict(m_rich, terms = c("year_trt", "trt")) %>%
  as_tibble() %>%
  mutate(
    trt_lab = factor(
      group,
      levels = c("Control", "N", "P", "K"),
      labels = c("Control", "+N", "+P", "+K")
    )
  )

dark2_cols <- brewer.pal(3, "Dark2")

```

```

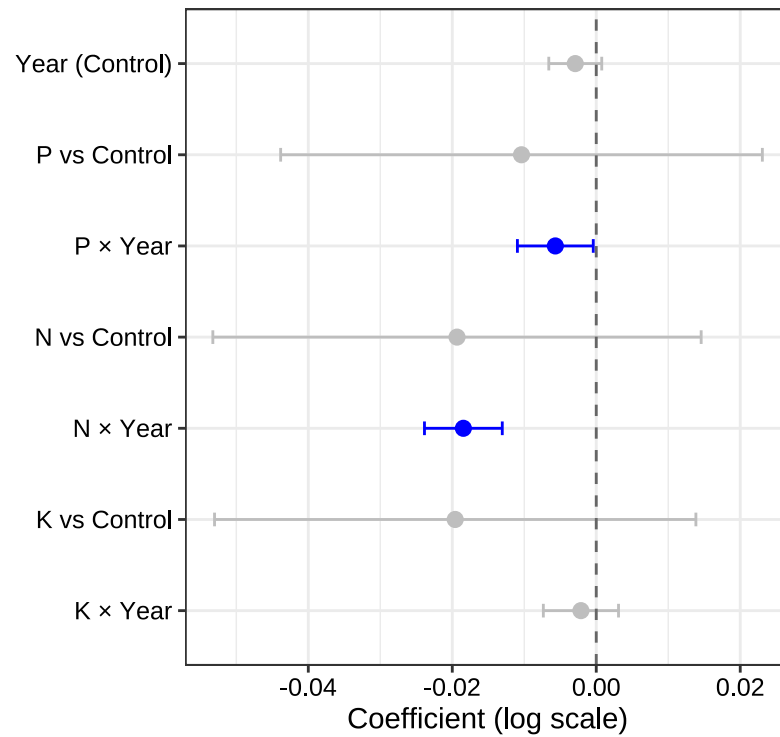
col_vec <- c(
  "Control" = "grey40",
  "+N"      = dark2_cols[1],
  "+P"      = dark2_cols[2],
  "+K"      = dark2_cols[3]
)

p_rich_b <- ggplot(pred_rich) +
  geom_line(aes(x = x, y = predicted,
               colour = trt_lab, group = trt_lab),
            size = 1.5) +
  geom_line(aes(x = x, y = conf.low,
               colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  geom_line(aes(x = x, y = conf.high,
               colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  theme_bw(base_size = 14) +
  labs(
    x = "Year",
    y = "Predicted species richness",
    colour = "Treatment:"
  ) +
  scale_colour_manual(values = col_vec) +
  theme(
    plot.title = element_blank(),
    legend.position = "bottom",
    axis.text = element_text(size = 12, color = "black")
  )

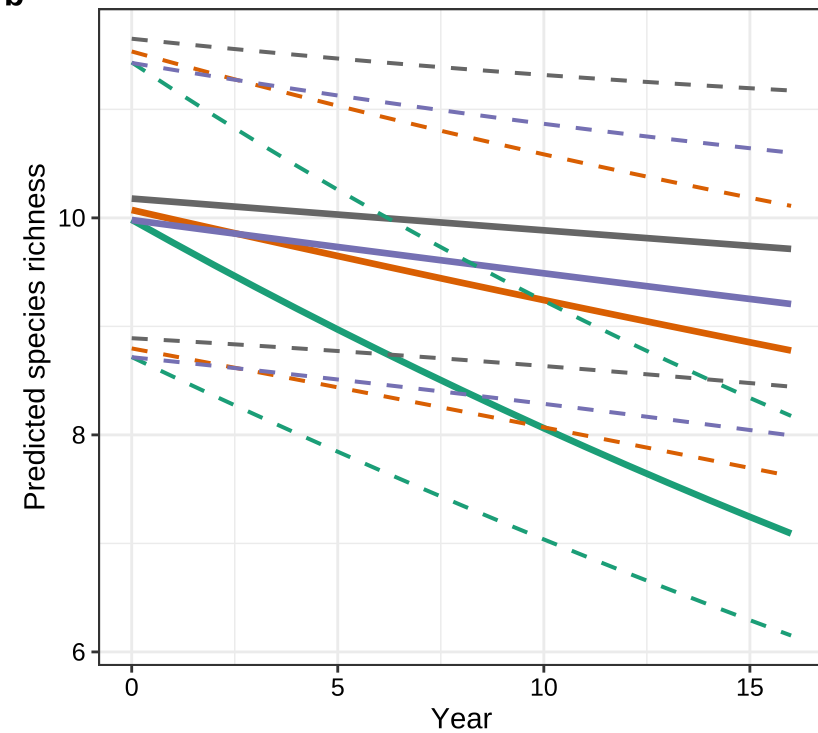
p_rich <- p_rich_a + p_rich_b +
  plot_layout(ncol = 2, widths = c(1, 1.2)) +
  plot_annotation(tag_levels = "a") &
  theme(plot.tag = element_text(face = "bold", size = 16))

p_rich

```

a

Significance: ● negative ● non-significant

b

Treatment: — Control — +N — +P — +K

1.2.4 Model and coefficient panel for dominant-species share

Dominant-species share is transformed to the open interval (0, 1) and modelled with a beta generalized linear mixed-effects model.

```
## ---- PREP DOMINANT SHARE FOR BETA MODEL ----

# Smithson & Verkuilen transform or simple clipping to (0,1):
n_obs <- nrow(df_dom)
df_dom <- df_dom %>%
```

```

mutate(
  dom_share_beta = (dom_share * (n_obs - 1) + 0.5) / n_obs
  # alternatively:
  # dom_share_beta = pmin(pmax(dom_share, 1e-4), 1 - 1e-4)
)

## ---- DOMINANT SHARE MODEL ----

m_dom <- glmmTMB(
  dom_share_beta ~ trt * year_trt +
    (1 | site_code),
  data = df_dom,
  ziformula = ~ 1, # allow extra zeros if present
  family = beta_family(link = "logit")
)

## ---- COEFFICIENT FIGURE (DOM SHARE) ----

coefs_dom <- broom.mixed::tidy(m_dom, effects = "fixed", conf.int = TRUE)

coefs_dom2 <- coefs_dom %>%
  filter(term != "(Intercept)")

coefs_dom2$term <- dplyr::recode(
  coefs_dom2$term,
  "trtK" = "K vs Control",
  "trtN" = "N vs Control",
  "trtP" = "P vs Control",
  "year_trt" = "Year (Control)",
  "trtK:year_trt" = "K x Year",
  "trtN:year_trt" = "N x Year",
  "trtP:year_trt" = "P x Year"
)

coefs_dom2 %<>%
  mutate(sig.sign = purrr::map2_chr(estimate, p.value, function(x, p) {
    if (x < 0 & p < 0.05) {

```



```

    "negative"
  } else if (x > 0 & p < 0.05) {
    "positive"
  } else {
    "non-significant"
  }
}))

p_dom_a <- ggplot(coefs_dom2, aes(estimate, term)) +
  geom_point(size = 3, aes(colour = sig.sign)) +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high, colour = sig.sign),
    height = 0.15) +
  geom_vline(xintercept = 0, linetype = 2, color = "grey40") +
  scale_color_manual(values = c("grey", "red")) +
  theme_bw(base_size = 14) +
  labs(
    x = "Coefficient (logit scale)",
    y = "",
    colour = "Significance:"
  ) +
  theme(
    axis.text = element_text(size = 12, color = "black"),
    legend.position = "bottom"
  )

```

1.2.5 Prediction panel for dominant-species share

Model predictions are generated to show how the share of dominant species changes through time across treatments.

```

## ---- PREDICTION FIGURE (DOM SHARE) ----

pred_dom <- ggpredict(m_dom, terms = c("year_trt", "trt")) %>%
  as_tibble() %>%
  mutate(
    # back to original year for x-axis:
    x_year = x + mean(df_dom$year_trt, na.rm = TRUE),
    trt_lab = factor(
      group,

```

```

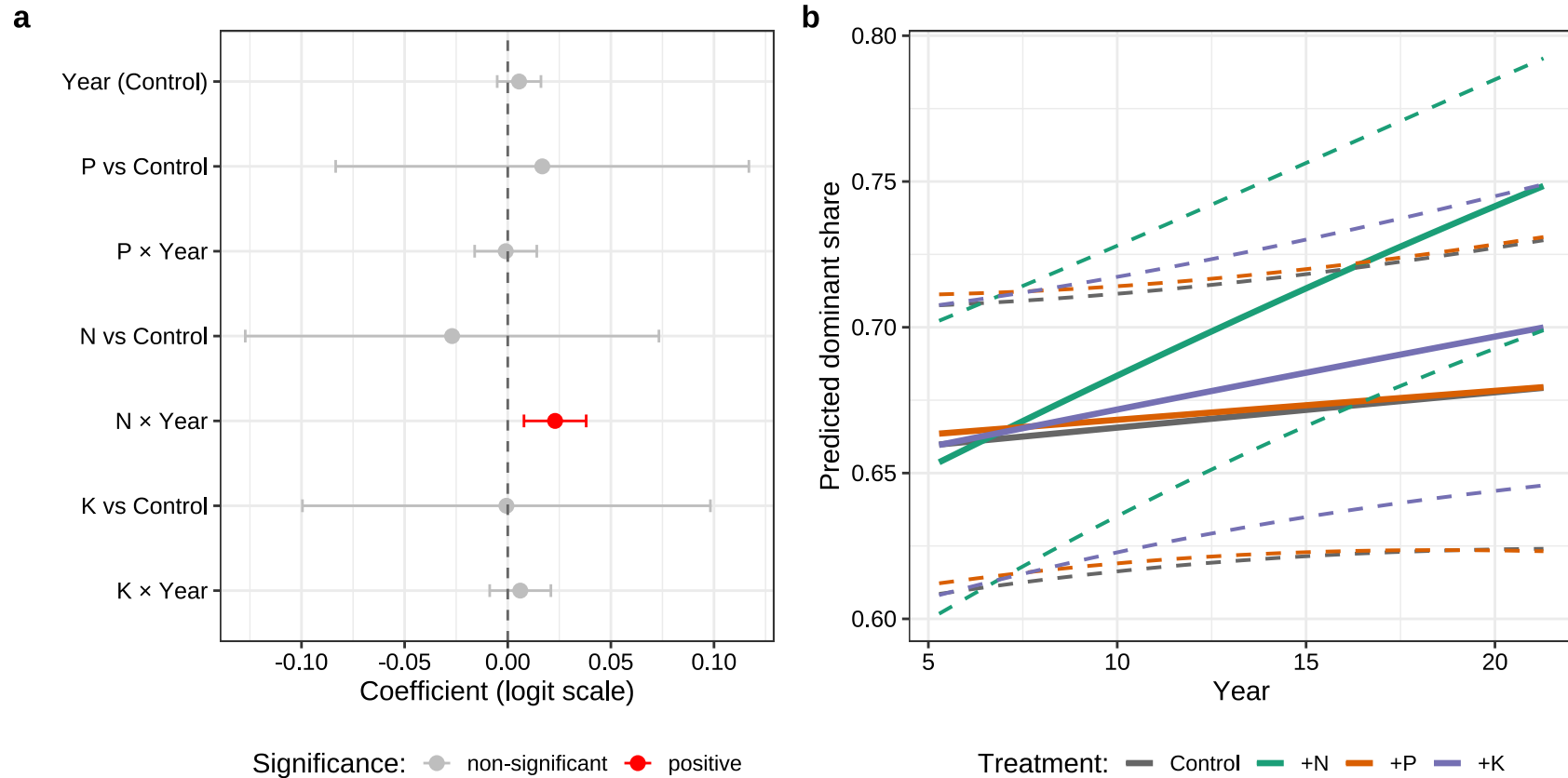
    levels = c("Control", "N", "P", "K"),
    labels = c("Control", "+N", "+P", "+K")
  )
)

p_dom_b <- ggplot(pred_dom) +
  geom_line(aes(x = x_year, y = predicted,
               colour = trt_lab, group = trt_lab),
            size = 1.5) +
  geom_line(aes(x = x_year, y = conf.low,
               colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  geom_line(aes(x = x_year, y = conf.high,
               colour = trt_lab, group = trt_lab),
            linetype = "dashed", size = 0.9) +
  theme_bw(base_size = 14) +
  labs(
    x = "Year",
    y = "Predicted dominant share",
    colour = "Treatment:"
  ) +
  scale_colour_manual(values = col_vec) +
  theme(
    plot.title = element_blank(),
    legend.position = "bottom",
    axis.text = element_text(size = 12, color = "black")
  )

p_dom <- p_dom_a + p_dom_b +
  plot_layout(ncol = 2, widths = c(1, 1.2)) +
  plot_annotation(tag_levels = "a") &
  theme(plot.tag = element_text(face = "bold", size = 16))

p_dom

```



1.3 Baseline soil N:P ratio and phosphorus stability responses

This section evaluates whether stability responses under phosphorus enrichment vary with baseline soil N:P ratio. This supplementary analysis is conducted separately from the main phosphorus enrichment-intensity models because baseline soil N:P and relative phosphorus enrichment intensity are strongly correlated.

1.3.1 Prepare phosphorus-enriched data and baseline N:P ratio

The analysis keeps phosphorus-enriched plots, adds baseline soil N and P concentrations, and calculates a scaled log-transformed N:P ratio.

```
# Reload the datasets required for this figure.
load(file = paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(file = paste0(data.path, "/df.tidy.Rdata"))

# Prepare the baseline soil N:P ratio analysis for phosphorus-enriched plots.
# Models including both phosphorus enrichment intensity and baseline N:P were considered initially.
# These predictors were strongly collinear because both depend on baseline soil P, leading to unstable parameter estimates.
# Therefore, this supplementary analysis focuses on baseline soil N:P only.
# Keep phosphorus-enriched plots only.

df_P_stabilities <- df.tidy.stabilities %>%
  filter(trt == "P")
# Add baseline soil N and P concentrations.
df_P_stabilities <- df_P_stabilities %>%
  left_join(
    df.tidy %>%
      select(site_code, block, trt, plot, P_added_proportion, pct_N, ppm_P) %>%
      distinct(),
    by = c("site_code", "block", "plot", "trt")
  )
# Calculate and scale baseline soil N:P ratio and temporal-window length.
df_P_stabilities <- df_P_stabilities %>%
  mutate(
    N_P_ratio = log(pct_N / ppm_P),
    logNP_sc = scale(N_P_ratio)[, 1],
    timescale = scale(end_year)[, 1],
  )
```

1.3.2 Fit stability-response models

Separate linear mixed-effects models are fitted for the four stability components, with baseline soil N:P ratio, temporal-window length, and their interaction as fixed effects and site as a random intercept.

```
m_P_func_var_det <- lmer(
  func_var_det ~
    logNP_sc      * timescale +
    (1 | site_code),
  data = df_P_stabilities
)

# Model for compositional temporal variability.
# Remove infinite compositional temporal-variability values before modelling.
df_P_stabilities_2 <- df_P_stabilities %>%
  filter(!is.infinite(comp_var))
m_P_comp_var <- lmer(
  comp_var ~
    logNP_sc      * timescale +
    (1 | site_code),
  data = df_P_stabilities_2 %>% na.omit()
)

# Model for extent of functional change.
df_P_stabilities_3 <- df_P_stabilities %>%
  filter(!is.infinite(func_change))
m_P_func_change <- lmer(
  func_change ~
    logNP_sc      * timescale +
    (1 | site_code),
  data = df_P_stabilities_3
)

# Model for extent of compositional change.
df_stabilities_4 <- df_P_stabilities %>%
  filter(!is.infinite(comp_change))
m_P_comp_change <- lmer(
  comp_change ~
    logNP_sc      * timescale +
    (1 | site_code),
  data = df_stabilities_4
)
```

1.3.3 Extract coefficients and display the figure

The coefficient figure shows whether baseline soil N:P ratio is associated with phosphorus-driven stability responses and whether these relationships change with temporal-window length.

```
get_coefs <- function(model, comp_label) {
  broom.mixed::tidy(model, effects = "fixed", conf.int = TRUE) %>%
    filter(term != "(Intercept)") %>%
    mutate(component = comp_label)
}

## Combine coefficient tables from the four stability-component models ----
coefs_all <- bind_rows(
  get_coefs(m_P_func_var_det, "func_var_det"),
  get_coefs(m_P_comp_var, "comp_var"),
  get_coefs(m_P_func_change, "func_change"),
  get_coefs(m_P_comp_change, "comp_change")
)

## Use readable predictor and stability-component names and classify significance ----
coefs_all <- coefs_all %>%
  mutate(
    term_pretty = recode(
      term,
      "logNP_sc" = "N:P ratio",
      "timescale" = "Years",
      "logNP_sc:timescale" = "N:P ratio × Years"
    ),
    # Set the order of predictor terms on the y-axis.
    term_pretty = factor(
      term_pretty,
      levels = c(
        "N:P ratio",
        "Years",
        "N:P ratio × Years"
      )
    ),
    # Set the order and labels of the stability components.
    component = factor(
```

```

    component,
    levels = c("func_var_det", "comp_var", "func_change", "comp_change"),
    labels = c("func_var_det", "comp_var", "func_change", "comp_change")
  ),
  # Classify estimates by direction and statistical significance.
  sig.sign = case_when(
    estimate > 0 & p.value < 0.05 ~ "positive",
    estimate < 0 & p.value < 0.05 ~ "negative",
    TRUE ~ "non-significant"
  ),
  sig.sign = factor(
    sig.sign,
    levels = c("negative", "non-significant", "positive")
  )
)

## Define colours consistent with the other coefficient plots ----
col_vec_sig <- c(
  "negative" = "blue",
  "non-significant" = "grey60",
  "positive" = "red"
)

## Plot coefficient estimates for all four stability components ----
p_P_NP_coefs <- ggplot(coefs_all,
  aes(x = estimate, y = term_pretty, colour = sig.sign)) +
  geom_point(size = 3) +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high),
    height = 0.15) +
  geom_vline(xintercept = 0, linetype = 2, colour = "grey40") +
  facet_wrap(~ component, nrow = 1, scales = "free_x") +
  scale_colour_manual(
    values = col_vec_sig,
    name = "Significance:"
  ) +
  theme_bw(base_size = 14) +
  labs(
    x = "The Coefficient Estimate",

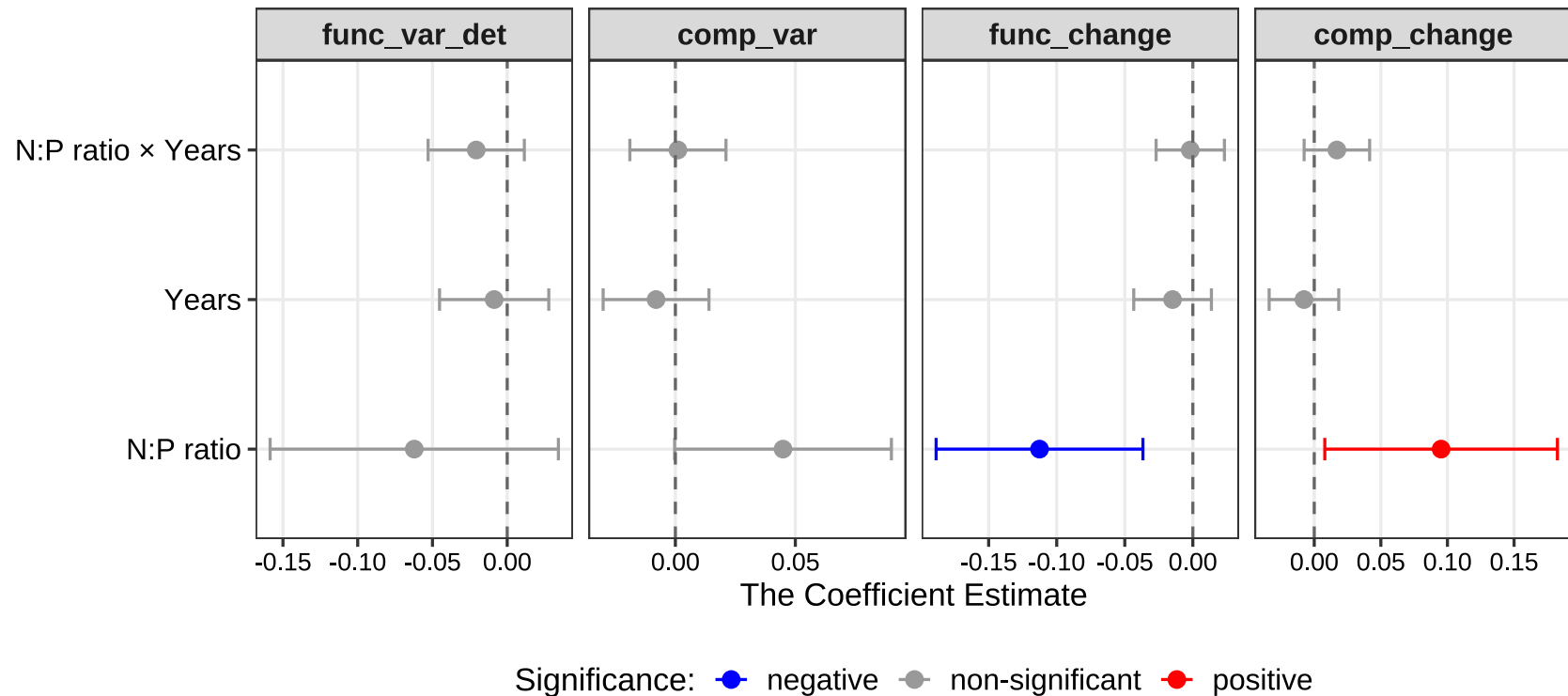
```

```

y = ""
) +
theme(
  axis.text.y      = element_text(size = 13, colour = "black"),
  axis.text.x      = element_text(size = 11, colour = "black"),
  legend.text      = element_text(size = 13),
  strip.text.x     = element_text(face = "bold", size = 13),
  legend.position  = "bottom",
  panel.grid.minor = element_blank()
)

```

p_P_NP_coefs



2. Directional biomass responses to enrichment intensity

This section examines directional biomass responses to nitrogen and phosphorus enrichment. Unlike the extent of functional change used in the main stability analyses, which is based on the absolute magnitude of biomass change, this analysis retains the direction of the biomass response using annual log response ratios between treatment and control plots.

2.1 Reconstruct biomass data and select control plots

The code recreates the biomass-control matching procedure used in the stability calculations. For each treatment plot, the corresponding control plot from the same block is attached so that annual log response ratios can be calculated.

```
# Reload the datasets required for this figure.
load(file = paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(file = paste0(data.path, "/df.tidy.Rdata"))

# Retain plots with biomass data.
df.tidy.mass <- df.tidy %>%
  filter(data.availability != "only coverage data") %>%
  select(site_code:trt, biomass.data) %>%
  rename(data = biomass.data)

# Identify all candidate control plots with biomass observations.
df.controls <- df.tidy.mass %>%
  unnest(cols = data) %>%
  ungroup() %>%
  filter(trt == "Control") %>%
  select(site_code, block, plot, year_trt) %>%
  distinct()

set.seed(1234)
df.controls.selected <- df.controls %>%
  arrange(site_code, block, desc(year_trt)) %>%
  group_by(site_code, block) %>%
  slice_head(n = 1) %>%
  ungroup() %>%
  select(-year_trt)

# Attach the selected control biomass time series to all plots in the same block.
```

```
df.tidy.mass.control <- df.tidy.mass %>%
  left_join(df.controls.selected, .) # same as in Rmd

df.tidy.mass.control <- df.tidy.mass.control %>%
  select(site_code, block, plot, data) %>%
  rename(control.plot = plot, control.data = data)

df.tidy.mass.2 <- df.tidy.mass %>%
  left_join(df.tidy.mass.control) %>%
  select(site_code, block, plot, control.plot, trt, data, control.data)
```

2.2 Calculate annual directional biomass log response ratios

Annual directional biomass responses are calculated as log response ratios of total biomass between treatment and control plots for overlapping post-treatment years.

```
df.lrr.dir <- df.tidy.mass.2 %>%
  filter(trt %in% c("N", "P")) %>%
  mutate(
    results = map2(
      .x = data,
      .y = control.data,
      .f = function(dat1, dat2) {
        # Combine treatment and control; keep overlapping years and year_trt >= 1
        dat <- dat1 %>%
          mutate(g = "experimental") %>%
          bind_rows(dat2 %>% mutate(g = "control")) %>%
          select(year_trt, g, mass) %>%
          filter(
            year_trt %in% intersect(dat1$year_trt, dat2$year_trt),
            year_trt >= 1
          )

        # Total biomass per year per group
        dat.nested <- dat %>%
          group_by(year_trt, g) %>%
          summarise(total_mass = sum(mass), .groups = "drop")
      }
    )
  )
```

```

    # Log response ratio per year
    lrr.total.mass <- dat.nested %>%
      pivot_wider(names_from = g, values_from = total_mass) %>%
      mutate(lrr = log(experimental / control))

    return(lrr.total.mass)
  }
)
) %>%
select(site_code, block, plot, trt, results) %>%
unnest(cols = results)

df.lrr.dir <- na.omit(df.lrr.dir) # removed one record
df.lrr.dir <- df.lrr.dir %>%
  filter(is.finite(lrr)) # removed records with infinite LRR (zero control biomass)

# Add relative enrichment intensity for the corresponding nutrient treatment.
df.lrr.dir.N <- df.lrr.dir %>%
  filter(trt == "N") %>%
  left_join(
    df.tidy %>%
      select(site_code, block, plot, trt, N_added_proportion) %>%
      distinct(),
    by = c("site_code", "block", "plot", "trt")
  ) %>%
  mutate(
    Ri_raw = N_added_proportion,
    Ri      = log(Ri_raw),
    Ri_sc   = scale(Ri)[, 1]
  )

```

2.3 Fit nitrogen and phosphorus directional-response models

Separate models are fitted for nitrogen and phosphorus enrichment, testing the effects of relative enrichment intensity, year, and their interaction on directional biomass responses.

```

m_LRR_N <- lmer(
  lrr ~ Ri_sc * year_trt + (1 | site_code),
  data = df.lrr.dir.N
)

df.lrr.dir.P <- df.lrr.dir %>%
  filter(trt == "P") %>%
  left_join(
    df.tidy %>%
      select(site_code, block, plot, trt, P_added_proportion) %>%
      distinct(),
    by = c("site_code", "block", "plot", "trt")
  ) %>%
  mutate(
    Ri_raw = P_added_proportion,
    Ri      = log(Ri_raw),
    Ri_sc   = scale(Ri)[, 1]
  )

m_LRR_P <- lmer(
  lrr ~ Ri_sc * year_trt + (1 | site_code),
  data = df.lrr.dir.P
)

```

2.4 Display directional biomass-response predictions

Predicted biomass log response ratios are shown for low, medium, and high relative enrichment intensities through time.

```

pred_N <- ggpredict(
  m_LRR_N,
  terms = c("year_trt [all]", "Ri_sc [-1, 0, 1]")
) %>%
  as_tibble() %>%
  mutate(
    Intensity = case_when(
      group == "-1" ~ "Low intensity",
      group == "0"  ~ "Medium intensity",

```

```

      group == "1" ~ "High intensity"
    ),
    Intensity = factor(
      Intensity,
      levels = c("Low intensity", "Medium intensity", "High intensity")
    )
  )

pred_P <- ggpredict(
  m_LRR_P,
  terms = c("year_trt [all]", "Ri_sc [-1, 0, 1]")
) %>%
  as_tibble() %>%
  mutate(
    Intensity = case_when(
      group == "-1" ~ "Low intensity",
      group == "0" ~ "Medium intensity",
      group == "1" ~ "High intensity"
    ),
    Intensity = factor(
      Intensity,
      levels = c("Low intensity", "Medium intensity", "High intensity")
    )
  )

col_vec <- c(
  "Low intensity" = "#FDE725FF",
  "Medium intensity" = "#21908CFF",
  "High intensity" = "#440154FF"
)
fill_vec <- col_vec

plot_pred <- function(pred_data) {
  ggplot(pred_data,
    aes(x = x, y = predicted,
        colour = Intensity, group = Intensity)) +
  geom_line(size = 1.25) +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high, fill = Intensity),

```

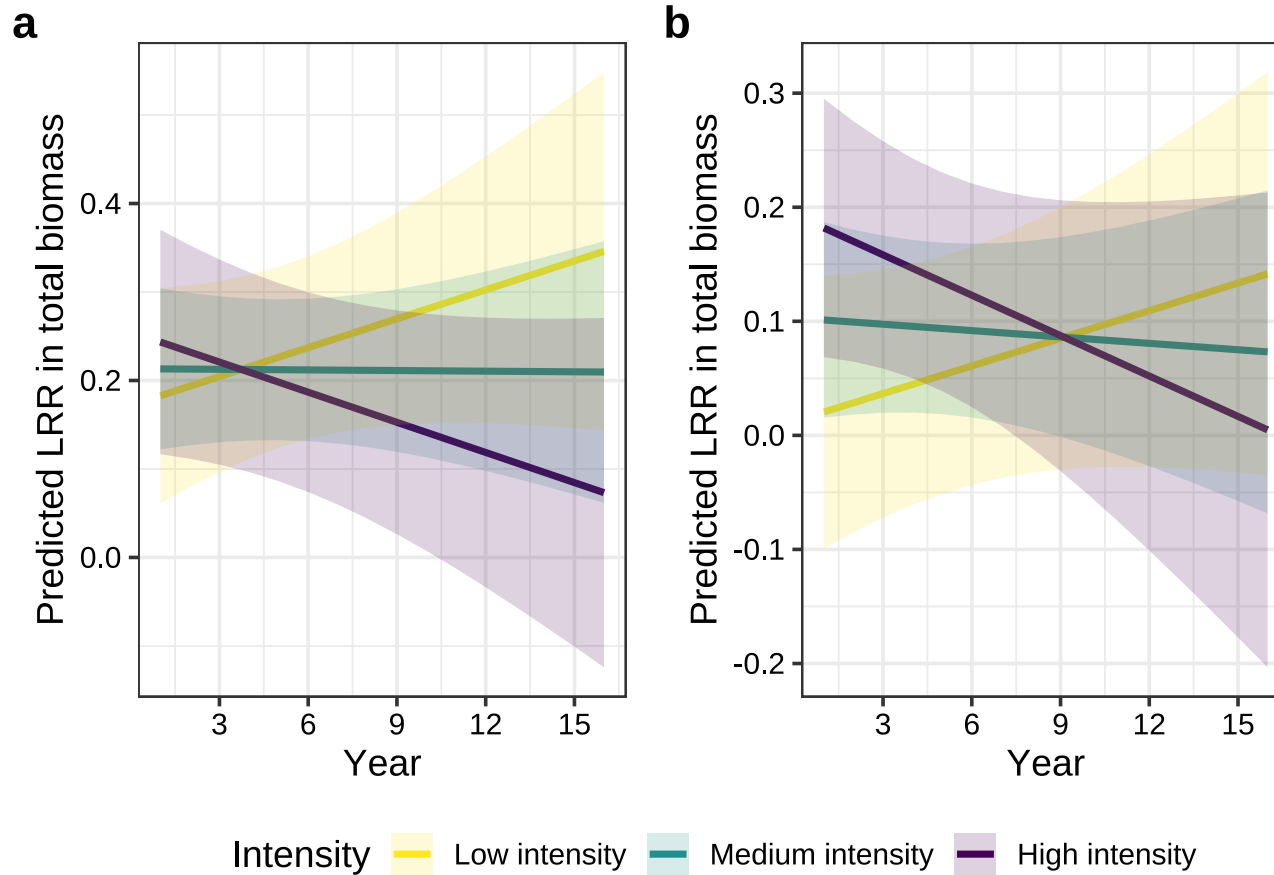
```

      alpha = 0.18, colour = NA) +
scale_x_continuous(breaks = c(0, 3, 6, 9, 12, 15)) +
scale_colour_manual(values = col_vec) +
scale_fill_manual(values = fill_vec) +
labs(
  x = "Year",
  y = "Predicted LRR in total biomass",
  title = NULL,
  colour = "Intensity",
  fill   = "Intensity"
) +
theme_bw(base_size = 14) +
theme(
  axis.text      = element_text(color = "black"),
  legend.position = "bottom",
  plot.title     = element_blank()
)
}

# Keep legends in both panels so patchwork can collect them into a single guide.
p_N_with_legend <- plot_pred(pred_N)
p_P_with_legend <- plot_pred(pred_P)
p_lrr_pred <- p_N_with_legend + p_P_with_legend +
  plot_annotation(
    tag_levels = 'a' # This adds A, B, C...
  ) +
  plot_layout(guides = 'collect') & theme(plot.tag = element_text(face = 'bold'), legend.position = 'bottom')

p_lrr_pred

```



3. Site-level baseline nutrients and enrichment intensity

This section maps the geographical distribution of baseline soil nutrient concentrations and relative enrichment intensities across the NutNet sites used in this study. The original script also created an Excel file containing nutrient levels and enrichment intensities; that file-writing step has been removed here because this Rmd focuses only on figure reproduction.

3.1 Prepare nutrient and enrichment-intensity data

The analysis extracts baseline soil nutrient concentrations and relative enrichment-intensity values for nitrogen, phosphorus, and potassium treatments.

```
# Reload the datasets required for this figure.
# Extract site, plot, baseline nutrient, and enrichment-intensity information.
load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))
df_nutrients <- df.tidy %>%
  select(site_code, block, plot, trt, latitude, longitude,
         pct_N, ppm_P, ppm_K, contains("proportion"), ) %>%
  distinct()

df.tidy.N <- filter(df_nutrients, trt=="N") %>% select(-ppm_P, -ppm_K, -P_added_proportion, -K_added_proportion)
df.tidy.P <- filter(df_nutrients, trt=="P") %>% select(-pct_N, -ppm_K, -N_added_proportion, -K_added_proportion)
df.tidy.K <- filter(df_nutrients, trt=="K") %>% select(-pct_N, -ppm_P, -N_added_proportion, -P_added_proportion)
# remove infinite values (if any)
df.tidy.N <- df.tidy.N %>%
  filter(is.finite(pct_N) | is.finite(N_added_proportion))
df.tidy.P <- df.tidy.P %>%
  filter(is.finite(ppm_P) | is.finite(P_added_proportion))
df.tidy.K <- df.tidy.K %>%
  filter(is.finite(ppm_K) | is.finite(K_added_proportion))

df.tidy.N$pct_N %>% range()

## [1] 0.009 1.281

df.tidy.P$ppm_P %>% range()

## [1] 5 254

df.tidy.K$ppm_K %>% range()

## [1] 18 1414
```



```
# Calculate log-transformed and scaled relative enrichment intensity for each nutrient.
df.tidy.N <- df.tidy.N %>%
  mutate(
    Ri_raw = N_added_proportion,
    Ri      = log(Ri_raw),
    Ri_sc   = scale(Ri)[, 1]
  )
df.tidy.P <- df.tidy.P %>%
  mutate(
    Ri_raw = P_added_proportion,
    Ri      = log(Ri_raw),
    Ri_sc   = scale(Ri)[, 1]
  )
df.tidy.K <- df.tidy.K %>%
  mutate(
    Ri_raw = K_added_proportion,
    Ri      = log(Ri_raw),
    Ri_sc   = scale(Ri)[, 1]
  )
```

3.2 Summarise values at the site level

Plot-level baseline nutrient concentrations and enrichment intensities are averaged within each site before mapping.

```
df.tidy.N.site <- df.tidy.N %>%
  group_by(site_code) %>%
  summarise(
    latitude = mean(latitude),
    longitude = mean(longitude),
    pct_N = mean(pct_N, na.rm = TRUE),
    Ri_sc = mean(Ri_sc, na.rm = TRUE)
  ) %>%
  ungroup()

df.tidy.P.site <- df.tidy.P %>%
  group_by(site_code) %>%
  summarise(
```

```

    latitude = mean(latitude),
    longitude = mean(longitude),
    ppm_P = mean(ppm_P, na.rm = TRUE),
    Ri_sc = mean(Ri_sc, na.rm = TRUE)
  ) %>%
ungroup()

df.tidy.K.site <- df.tidy.K %>%
  group_by(site_code) %>%
  summarise(
    latitude = mean(latitude),
    longitude = mean(longitude),
    ppm_K = mean(ppm_K, na.rm = TRUE),
    Ri_sc = mean(Ri_sc, na.rm = TRUE)
  ) %>%
ungroup()

```

3.3 Prepare spatial objects

The site-level tables are converted to spatial objects and combined with a world map for plotting.

```

# Load a world map for plotting site locations.
world <- ne_countries(scale = "medium", returnclass = "sf")

# Convert site-level tables to spatial objects.
sites_sf_N <- st_as_sf(df.tidy.N.site,
  coords = c("longitude", "latitude"), crs = 4326)
sites_sf_P <- st_as_sf(df.tidy.P.site,
  coords = c("longitude", "latitude"), crs = 4326)
sites_sf_K <- st_as_sf(df.tidy.K.site,
  coords = c("longitude", "latitude"), crs = 4326)

```

3.4 Map relative enrichment intensity

These maps show the site-level relative enrichment intensity for nitrogen, phosphorus, and potassium.

```

# Map site-level relative nitrogen enrichment intensity.
p.map.N.Ri_sc <- ggplot(data = world) +
  geom_sf(fill = "gray90", color = "gray40", linewidth = 0.1) +
  geom_sf(
    data = sites_sf_N,
    aes(fill = Ri_sc),
    color = "white",
    size = 2,
    shape = 21,
    stroke = 0.2
  ) +
  scale_fill_gradient(
    low = "#E6F5EE",
    high = "#1B9E77",
    name = "N enrichment"
  ) +
  theme_minimal() +
  labs(title = "N perturbation intensity (Ri_sc)") +
  theme(
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.text = element_blank(),
    axis.title = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    plot.subtitle = element_text(hjust = 0.5, size = 12),
    plot.caption = element_text(hjust = 0.5, size = 10)
  )

# Map site-level relative phosphorus enrichment intensity.
p.map.P.Ri_sc <- ggplot(data = world) +
  geom_sf(fill = "gray90", color = "gray40", linewidth = 0.1) +
  geom_sf(
    data = sites_sf_P,
    aes(fill = Ri_sc),
    color = "white",
    size = 2,

```

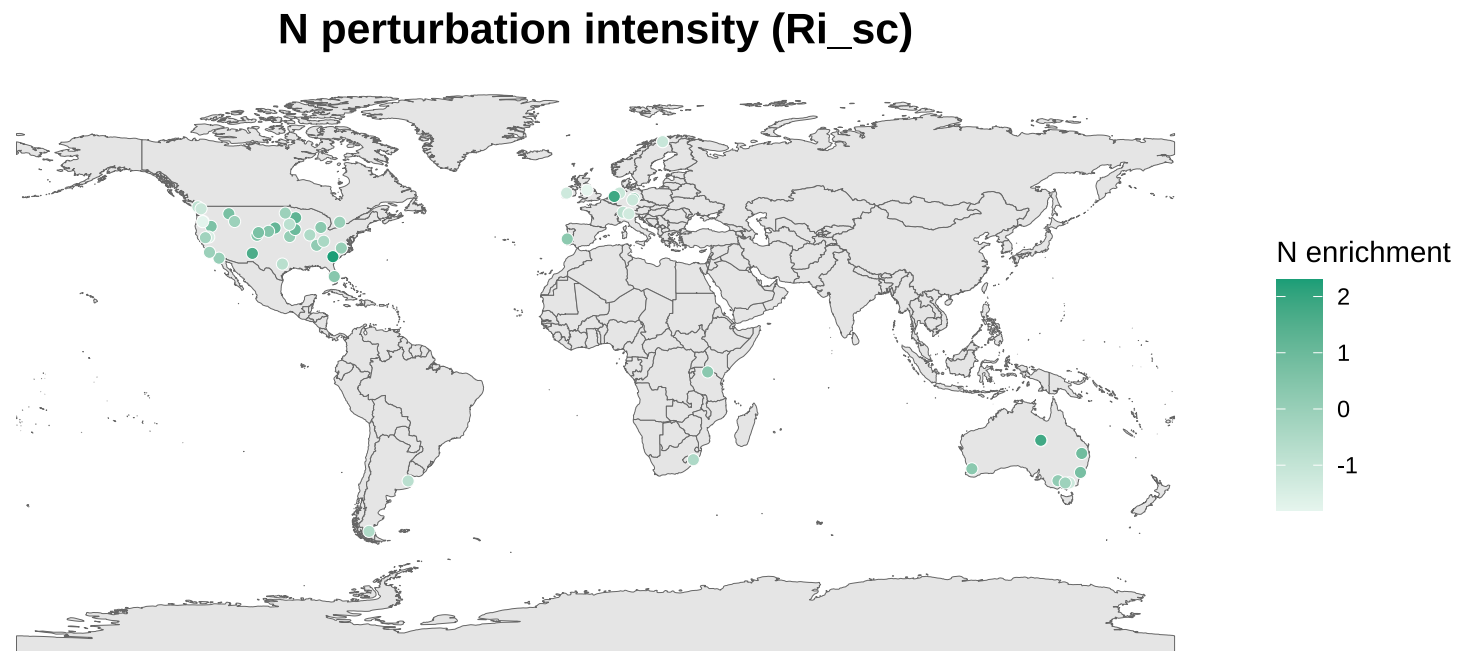
```

    shape = 21,
    stroke = 0.2
  ) +
  scale_fill_gradient(
    low = "#FEE8D8",
    high = "#D95F02",
    name = "P enrichment"
  ) +
  theme_minimal() +
  labs(title = "P perturbation intensity (Ri_sc)") +
  theme(
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.text = element_blank(),
    axis.title = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    plot.subtitle = element_text(hjust = 0.5, size = 12),
    plot.caption = element_text(hjust = 0.5, size = 10)
  )

# Map site-level relative potassium enrichment intensity.
p.map.K.Ri_sc <- ggplot(data = world) +
  geom_sf(fill = "gray90", color = "gray40", linewidth = 0.1) +
  geom_sf(
    data = sites_sf_K,
    aes(fill = Ri_sc),
    color = "white",
    size = 2,
    shape = 21,
    stroke = 0.2
  ) +
  scale_fill_gradient(
    low = "#ECEAF6",
    high = "#7570B3",
    name = "K enrichment"
  ) +

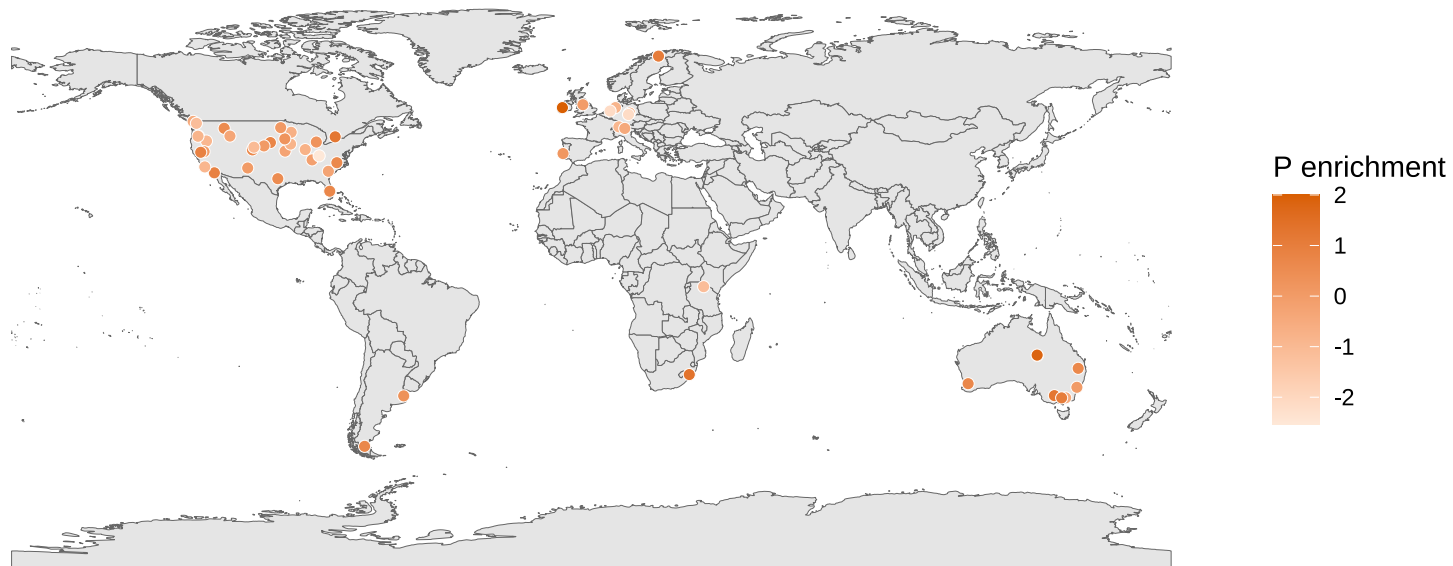
```

```
theme_minimal() +  
labs(title = "K perturbation intensity (Ri_sc)") +  
theme(  
  panel.background = element_rect(fill = "white", color = NA),  
  panel.grid.major = element_blank(),  
  panel.grid.minor = element_blank(),  
  axis.text = element_blank(),  
  axis.title = element_blank(),  
  axis.ticks = element_blank(),  
  plot.title = element_text(hjust = 0.5, face = "bold", size = 16),  
  plot.subtitle = element_text(hjust = 0.5, size = 12),  
  plot.caption = element_text(hjust = 0.5, size = 10)  
)  
  
print(p.map.N.Ri_sc)
```



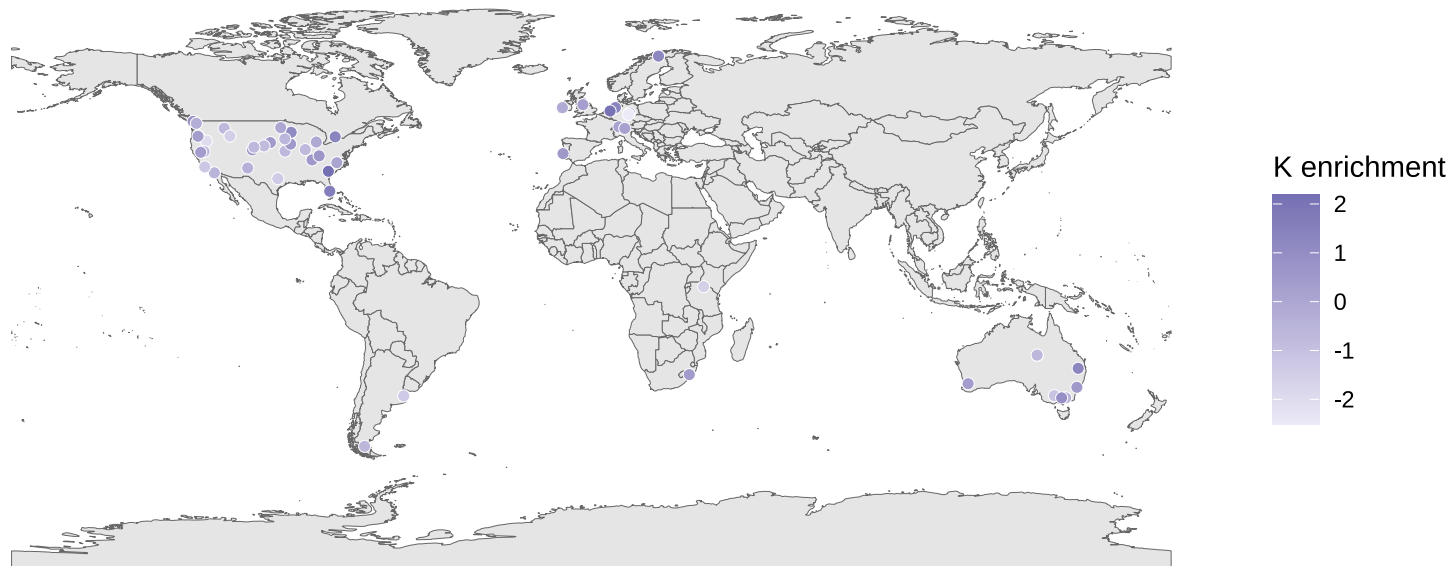
```
print(p.map.P.Ri_sc)
```

P perturbation intensity (Ri_sc)



```
print(p.map.K.Ri_sc)
```

K perturbation intensity (Ri_sc)



4. Sensitivity analysis for single stability responses

This final section repeats the single-stability-response models using only the subset of plots included in the dimensionality analysis. This provides a sensitivity check for whether the main results for individual stability components are qualitatively robust to restricting the dataset to the plots with sufficiently long time series used for dimensionality estimation.

4.1 Prepare the dimensionality-analysis subset

```
# repeat codes in step 04b to get plots included in dimensionality analysis
load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))
df.full <- df.tidy.stabilities %>%
  left_join(df.tidy %>% select(-contains("data")))
rm(df.tidy, df.tidy.stabilities)

df.full %<>%
  select(site_code:end_year, matches("_added_proportion|change|var"))
# Use the four stability components analysed in the manuscript and exclude
# the non-detrended functional temporal variability metric.
sta.names <- c("func_var_det", "func_change", "comp_var", "comp_change")

# For the PCA analysis, all stability components need to be available for each row
# no NA, no infinite
# remove the relevant cases
df.full <- df.full %>%
  filter_all(all_vars(!is.infinite(.))) %>%
  filter_all(all_vars(!is.na(.)))

df.stability <- df.full

# scale stability components
df.stability %<>% mutate(across(one_of(sta.names), scale))

# make a meta data for multidimensionality analysis
trt.set <- c("N", "P", "K")
final.year.set <- 9 # original using 5:9
```

```

df.meta <- crossing(trt.set, final.year.set)

## add data, nested by year

df.meta <- df.meta %>%
  mutate(data = map2(
    trt.set, final.year.set,
    function(x1, x2) {
      plots.to.keep <- df.stability %>%
        filter(
          trt == x1,
          end_year == x2
        ) %>%
        select(site_code, block, trt, plot)
      subdat <- df.stability %>%
        semi_join(plots.to.keep) %>%
        filter(
          trt == x1,
          end_year <= x2
        )
      return(subdat)
    }
  ))
df.meta %<>% ungroup()

## add data, nested by perturbation intensity and end_year

df.meta <- df.meta %>%
  mutate(data2 = map2(
    trt.set, data,
    function(x1, dat) {
      df.pert <- dat %>%
        select(site_code, block, trt, plot, paste0(x1, "_added_proportion")) %>%
        distinct() %>%
        set_colnames(c("site_code", "block", "trt", "plot", "PI"))
      df.pert %<>%
        mutate(PI.group = ifelse(PI > median(PI), "High", "Low"))
      dat %<>% left_join(df.pert)
    }
  ))

```

```

    dat.nested <- dat %>%
      group_by(end_year, PI.group) %>%
      nest() %>%
      ungroup()
    return(dat.nested)
  }
))

df.meta %<>%
  select(-data) %>%
  unnest(cols = data2)
df_plots_dimensionality <- df.meta %>%
  select(trt.set, PI.group, data) %>%
  unnest(cols = data) %>%
  select(trt.set, PI.group, site_code, block, plot)

#####

# Reload the datasets required for this sensitivity analysis.
load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))

df.full <- df.tidy.stabilities %>%
  left_join(df.tidy %>% select(-contains("data")))
rm(df.tidy, df.tidy.stabilities)

df.full %<>%
  select(site_code:end_year, matches("_added_proportion|change|var"))
# Use the four stability components analysed in the manuscript and exclude
# the non-detrended functional temporal variability metric.
sta.names <- c("func_var_det", "func_change", "comp_var", "comp_change")

df.full <- df.full %>%
  semi_join(df_plots_dimensionality, by = c("site_code", "block", "plot"))

# Scale stability components after replacing infinite values with missing values.

```

```

# This helper safely handles infinite values before scaling.
new.scale <- function(x) {
  x[is.infinite(x)] <- NA
  y <- scale(x)
}

df.full <- df.full %>%
  mutate(
    func_var = new.scale(func_var),
    func_var_det = new.scale(func_var_det),
    comp_var = new.scale(comp_var),
    func_change = new.scale(func_change),
    comp_change = new.scale(comp_change)
  )

# Build a nested dataset for each nutrient treatment and stability component.
df.meta.N.P.K <- data.frame(trt = c("N", "P", "K")) %>%
  mutate(pert.name = paste0(trt, "_added_proportion")) %>%
  expand_grid(sta.names)

df.N.P.K.nested <- df.meta.N.P.K %>%
  mutate(
    data = pmap(list(trt, pert.name, sta.names), function(x1, x2, x3) {
      output <- df.full %>%
        filter(trt == x1) %>%
        select(one_of(c("site_code", "block", "plot", "end_year", x2, x3))) %>%
        set_colnames(c("site_code", "block", "plot", "end_year", "perturbation_intensity", "stability"))
    })
  )

# Some stability values are missing because not all plots contain both cover and biomass data.
# Remove missing values before fitting the models.
# Remove missing stability values before model fitting.
df.N.P.K.nested <- df.N.P.K.nested %>%
  mutate(data = map(data, na.omit))
# Infinite stability values are also removed.
# Remove infinite stability values.
df.N.P.K.nested <- df.N.P.K.nested %>%
  mutate(data = map(data, function(df1) df1 %>% filter(!is.infinite(stability))))

```

4.2 Fit sensitivity models

For each nutrient treatment and stability component, the model tests the effects of relative enrichment intensity, temporal-window length, and their interaction, with site as a random intercept.

```
df.models.N.P.K <- df.N.P.K.nested %>%
  mutate(mod.out = map(data, function(dat) {
    # log transform perturbation intensity
    dat %<>% mutate(log.pert = log(perturbation_intensity))
    dat %<>% na.omit()
    mod.dat <- dat
    mod.dat %<>% mutate(across(c(end_year, log.pert), scale))
    formula_ <- stability ~ log.pert + end_year + log.pert:end_year + (1 | site_code)
    mod <- lmer(formula = formula_, data = mod.dat)
    return(mod)
  })))
```

4.3 Extract model coefficients

Coefficient estimates, confidence intervals, and P values are extracted and organised for plotting.

```
df.models.N.P.K.coefficients <- df.models.N.P.K %>%
  mutate(mod.coefficient = map(mod.out, function(mod) {
    # Extract coefficient estimates from the model summary.
    output <- summary(mod)$coefficients
    coeff <- output %>% as.data.frame()
    coeff %<>% mutate(predictor = rownames(output))
    coeff %<>% select(predictor, Estimate)
    # Extract confidence intervals for fixed effects.
    # Use profile confidence intervals from lme4.
    cfs <- confint.merMod(mod)
    cfs %<>% as.data.frame()
    cfs %<>% mutate(predictor = rownames(cfs))
    cfs %<>% rename(lcf = `2.5 %`, ucf = `97.5 %`)
    cfs %<>% filter(!str_detect(predictor, ".sig"))
    # Extract Satterthwaite F-test P values from lmerTest.
    df.p.value <- anova(mod) %>% as.data.frame()
    df.p.value %<>% select(NumDF: `Pr(>F)`)
  })))
```

```

df.p.value %<>% mutate(predictor = rownames(df.p.value))

# Classify coefficients by sign and statistical significance.
coeff <- coeff %>%
  left_join(cfs) %>%
  left_join(df.p.value)
coeff %<>% filter(predictor != "(Intercept)")
coeff %<>%
  mutate(sig.sign = map2_chr(Estimate, `Pr(>F)`, function(x, p) {
    if (x < 0 & p < 0.05) {
      out <- "negative"
    } else if (x > 0 & p < 0.05) {
      out <- "positive"
    } else {
      out <- "non-significant"
    }
    return(out)
  }))
return(coeff)
}))

df.models.N.P.K.coefficients %<>%
  mutate(model.ID = 1:nrow(df.models.N.P.K.coefficients)) %>%
  select(-pert.name, -data, -mod.out) %>%
  select(model.ID, everything()) %>%
  unnest(cols = mod.coefficient)

```

4.4 Display sensitivity-analysis figure

The coefficient figure shows whether the main single-stability-response results are qualitatively robust when only the dimensionality-analysis subset is used.

```

df.coefficients <- df.models.N.P.K.coefficients %>%
  # Keep only the four stability components reported in the manuscript.
  filter(sta.names %in% c("func_var_det", "func_change", "comp_var", "comp_change")) %>%
  mutate(

```

```

predictor = str_replace_all(predictor, "end_year", "years"),
predictor = str_replace_all(predictor, "log.pert", "perturbation"),
# Use the same stability names and order as in the manuscript.
sta.names = factor(
  sta.names,
  levels = c("func_var_det", "func_change", "comp_var", "comp_change"),
  labels = c(
    "Functional
temporal
variability",
    "Extent of
functional
change",
    "Compositional
temporal
variability",
    "Extent of
compositional
change"
  )
),
trt = factor(trt, levels = c("N", "P", "K")),
predictor = factor(
  predictor,
  levels = c("years", "perturbation", "perturbation:years")
)
)

p1 <- df.coefficients %>%
# filter(str_detect(sta.names, "functional")) %>%
ggplot(aes(x = predictor, y = Estimate, color = sig.sign)) +
geom_point() +
geom_errorbar(aes(ymin = lcf, ymax = ucf), width = .2) +
facet_grid(trt ~ sta.names) +
coord_flip() +
geom_hline(yintercept = 0, col = "grey", linetype = "dashed", size = 0.3) +
scale_color_manual(values = c("blue", "grey", "red")) +
theme_linedraw() +

```

```
theme(
  strip.background = element_rect(fill = "gray90"),
  strip.text = element_text(color = "black", size = 8),
  panel.grid = element_blank(),
  axis.text.x = element_text(size = 9, color = "black", angle = 90),
  axis.text.y = element_text(size = 10, color = "black"),
  axis.title.x = element_text(size = 12),
  axis.title.y = element_blank(),
  plot.title = element_text(face = "bold", color = "blue"),
  legend.position = "bottom",
  legend.title = element_blank()
) +
  xlab("Predictors") +
  ylab("The Coefficient Estimate")

trt.labs <- c("+N", "+P", "+K")
names(trt.labs) <- c("N", "P", "K")

# Apply treatment labels and display the plot.
(p.sensitivity <- p1 + facet_grid(
  trt ~ sta.names,
  labeller = labeller(trt = trt.labs)
))
```